Final Project Report

Motorcycle Helmet Detection System



Project Supervisor

Laraib Sana

Submitted By

S26PROJECT7DE39

Arbab Hassan BC230206680

Jalil Riaz BC230206460

Software Projects & Research Section,
Department of Computer Sciences,
Virtual University of Pakistan

CERTIFICATE

This is to certify that Arbab Hassan (BC230206680), Jalil Riaz (BC230206460) have worked on and completed their Software Project at Software & Research Projects Section, Department of Computer Sciences, Virtual University of Pakistan in partial fulfillment of the requirement for the degree of BS in Computer Sciences under my guidance and supervision.

In our opinion, it is satisfactory and up to the mark and therefore fulfills the requirements of BS in Computer Sciences.

Supervisor / Internal Examiner

Laraib Sana
Supervisor,
Software Projects & Research Section,
Department of Computer Sciences
Virtual University of Pakistan

_____
(Signature)

External Examiner/Subject Specialist

<<External Supervisor Name>>

_____
(Signature)

Accepted By:

_____
(For office use)

EXORDIUM

In the name of Allah, the Compassionate, the Merciful.

Praise be to Allah, Lord of Creation,
The Compassionate, the Merciful,
King of Judgment-day!

You alone we worship, and to You alone we pray for help,
Guide us to the straight path

The path of those who You have favored,

Not of those who have incurred Your wrath,
Nor of those who have gone astray.

DEDICATION

We dedicate this project to our families, teachers and fellow students whose encouragement supports our learning. We also dedicate this work to the people who promote responsible motorcycle riding and the consistent use of protective helmets.

The project reflects our effort to apply computer science to a practical visual recognition problem. Its contribution is a documented prototype that can support human review, while recognising that automated predictions can contain errors.

ACKNOWLEDGEMENT

We are grateful to Allah for the opportunity to complete this project. We thank our supervisor and teachers for their academic guidance, and our families and peers for their encouragement throughout the work.

We acknowledge the creators of the EdgeVision Dataset for making annotated motorcycle images available for research. Their contribution provided the data needed to study helmet and no-helmet detection in road scenes. The dataset attribution and licence are included in the references.

We also acknowledge the developers of Ultralytics, PyTorch, Gradio and the other software used in the project. Google Colab provided the computing environment for training the detector. These resources supported our work from data preparation through evaluation and application integration.

Preparing this project strengthened our understanding of image annotation, model training and the careful interpretation of detection results. We remain responsible for checking the implementation, reporting its measured performance accurately and recognising the conditions in which further testing is needed.

PREFACE

This report presents a motorcycle helmet detection system using YOLOv8m. It locates helmeted and unhelmeted heads in photographs and camera frames, and displays boxes, labels and confidence scores. The revised training addresses a weakness in the earlier model: many heads without helmets were missed. The updated app adds photo capture and continuous webcam input.

The workflow uses the EdgeVision Dataset and two target classes. It includes annotation conversion, data cleaning, separate training, validation and test splits, and training-time augmentation. The selected checkpoint is used by the Python app. The latest interface adds clear class colours, confidence controls and on-screen warnings while preserving the original model-loading order.

The completed training run stopped after 36 epochs, with the best validation checkpoint selected at epoch 16. On 348 held-out test images, the model achieved 90.83% mAP50. At the documented application threshold, no-helmet precision was 81.88% and recall was 85.29%. These measurements describe performance on the evaluated data; additional camera and road conditions require separate assessment.

The report follows the university's three-chapter structure. Chapter 1 sets out requirements and the development approach. Chapter 2 describes the design, diagrams and data flow. Chapter 3 explains implementation, training, evaluation and deployment. References and appendices provide supporting records and operating notes. Camera performance and final acceptance remain subject to testing on the demonstration laptop.

TABLE OF CONTENTS

Section	Page
Certificate	**2**
Exordium	**3**
Dedication	**4**
Acknowledgement	**5**
Preface	**6**
CHAPTER 1 — Gathering & Analyzing Info	**10**
1.1 Introduction	11
1.2 Purpose	11
1.3 Scope	11
1.4 Definitions, Acronyms and Abbreviations	12
1.5 Project Requirements	12
1.5.1 Functional Requirements	12
1.5.2 Non-Functional Requirements	13
1.6 Use Cases and Usage Scenarios	14
1.6.1 Use Case Diagrams	14
1.6.2 Usage Scenarios	15
1.7 Development Methodology	16
1.7.1 Chosen Methodology	16
1.7.2 Reasons for Chosen Methodology	16
1.7.3 Work Plan (Gantt Chart)	17
1.7.4 Project Schedule (Submission Calendar)	17

TABLE OF CONTENTS (continued)

Section	Page
CHAPTER 2 — Designing the Project	**18**
2.1 Introduction	19
2.2 Purpose	19
2.3 Scope	19
2.4 Definitions, Acronyms and Abbreviations	19
2.5 Architectural Representation (Architecture Diagram)	20
2.6 Dynamic Model: Sequence Diagrams	21
2.6.1 Image and Webcam Detection Sequence	21
2.6.2 Training and Evaluation Sequence	22
2.7 Object Model/Logical Model: Class Diagram	23
2.8 Database Model (Database Diagram)	24
2.9 Graphical User Interfaces	25

TABLE OF CONTENTS (continued)

Section	Page
CHAPTER 3 — Development	**26**
3.1 Development Plan (Architecture Diagram)	27
3.2 Dataset Acquisition and Class Conversion	28
3.3 Data Cleaning and Validation	29
3.4 Dataset Splitting and Training Exposure	30
3.5 Data Augmentation	31
3.6 Model Training and Checkpoint Selection	32
3.7 Evaluation Method and Results	33
3.8 Testing and Verification	34
3.9 Application Integration and Deployment	35
3.10 Limitations, Future Work and Conclusion	36
REFERENCES	**37**
APPENDIX	**38**
A Project Files and Reproducibility Record	38
B Core Application Code	39
C Operation and Demonstration Notes	40

CHAPTER 1

Gathering & Analyzing Info

1.1 Introduction

Helmet detection is an object detection task: the software must locate relevant head regions and classify them, rather than assign one label to an entire photograph. A road image may contain several riders with different helmet states. The system therefore returns a separate bounding box and prediction for each detected region.

The original main model used YOLOv8m with two classes. Its saved validation output showed no-helmet recall of 29.6%, indicating that many annotated unhelmeted heads were missed on that set. The revised project retains this model architecture, improves the data preparation and training workflow, and extends the application with live camera processing. [1]

1.2 Purpose

The purpose is to provide a reproducible training process and an application that displays helmet and no-helmet detections for human review. The main technical objective is to improve detection of unhelmeted heads without generating an unacceptable number of false detections. Separate precision and recall measurements are used to assess that objective.

1.3 Scope

The scope covers Drive dataset loading, ZIP extraction, class conversion, cleaning, grouped splits, training exposure balancing, augmentation, YOLOv8m training, evaluation and best.pt export. The app supports uploaded photos, camera photographs and continuous webcam frames. The existing YOLOv11 checkpoint remains a file-presence fallback.

The app displays Helmet/No Helmet boxes, confidence scores, counts and a visual warning for no-helmet detections. Automatic fines, identity recognition, plate recognition, sound alerts and persistent violation logging are not implemented. Camera generalisation needs local testing, and detected heads are not explicitly associated with particular motorcycles.

1.4 Definitions, Acronyms and Abbreviations

Term	Meaning in this project
YOLO	You Only Look Once, the object detection model family.
Bounding box	A rectangular region around a labelled or detected head.
Epoch	One pass through the configured training samples.
Checkpoint	A saved model state, such as best.pt or last.pt.
IoU	Intersection over Union: overlap area divided by union area.
Precision	Correct detections divided by all detections for a class.
Recall	Correct detections divided by all labelled objects of a class.
AP / mAP	Average Precision / mean AP across the output classes.
GPU / CPU	Graphics / central processing unit.
F1	Harmonic mean of precision and recall.

1.5 Project Requirements

Requirements cover the application used for demonstration and the training notebook used to create its weights. Each requirement below is tied to an observable result rather than a claim of perfect recognition.

1.5.1 Functional Requirements

ID	Required behaviour
FR01	Accept uploaded images and webcam photographs through Gradio.
FR02	Load best.pt first; use yolov11nbest.pt only if best.pt is absent.
FR03	Display colour-coded Helmet/No Helmet boxes, confidence scores and counts.
FR04	Prepare a reproducible two-class dataset from the full EdgeVision ZIP.
FR05	Train, select a validation checkpoint and evaluate a held-out test set.
FR06	Save training evidence; process live camera frames and show No Helmet warnings.

1.5.2 Non-Functional Requirements

ID	Requirement	Acceptance or verification
NFR01	Reproducibility	Save the seed, settings, class mapping, split and dataset fingerprint.
NFR02	Detection quality	Assess both no-helmet precision and recall against a 75% project target on the fixed test protocol.
NFR03	Usability	Provide Photo check and Live webcam tabs, clear labels and confidence controls.
NFR04	Portability	Keep inference in Python; verify the dependency environment on the intended demonstration laptop.
NFR05	Recoverability	Save training checkpoints to Drive and provide a resume option.
NFR06	Traceability	Retain dataset attribution, cleaning decisions and evaluation settings.
NFR07	Performance	Measure inference latency on the deployment device; avoid assuming GPU timings apply to CPU.

The reported held-out no-helmet precision and recall exceed the numerical target, but this does not establish the same performance for every road, camera angle or lighting condition. The evaluation uses a particular annotation set and matching protocol. Errors in source labels and related images across splits can affect the estimate.

The application should be used as a demonstration and review aid. Its current code contains no login, permissions system, persistent incident history or manual correction workflow. Those capabilities would require further development rather than configuration of the trained weights.

Installation must use a compatible Python, PyTorch, Ultralytics and Gradio environment. The supplied requirements file does not pin exact versions. The recorded training version is Ultralytics 8.3.162; preserving that version for initial model loading helps isolate compatibility issues. [1, 2]

1.6 Use Cases and Usage Scenarios

1.6.1 Use Case Diagrams

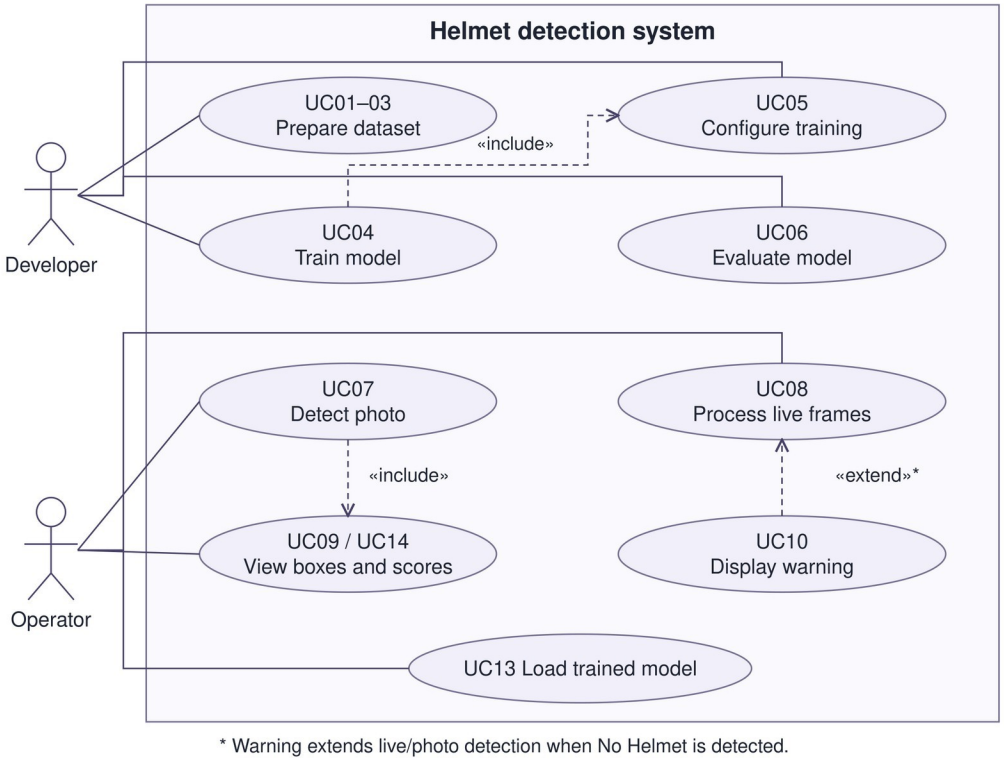


Figure 1.1 Application and training use cases

The SRS roles are retained: the developer prepares data, configures training and evaluates the model; the operator supplies a photograph or starts the camera and reviews results. Actors remain outside the system boundary. The YOLO model is an internal component, not a separate human actor. These roles do not imply login accounts.

The diagram follows the SRS use-case structure and IDs. UC10 represents the implemented visual warning. UC11 image archiving and UC12 sound alerts remain optional, unimplemented extensions. Detection requires a loaded checkpoint; an absence of boxes does not establish helmet compliance.

1.6.2 Usage Scenarios

Scenario	Steps and expected outcome
SRS UC07/09/14 Photo detection	Upload or capture a photo. predict() calls detect(), which returns labelled boxes, confidence, counts and timing. No Helmet detections produce a visual warning (UC10).
SRS UC08/10 Live detection	Start browser streaming or direct OpenCV capture. Each frame is detected and annotated. Stop using camera controls or Q/Esc in OpenCV. Camera performance needs local testing.
SRS UC13 Main checkpoint absent	At startup, best.pt is absent and yolov11nbest.pt exists. The app loads the fallback checkpoint. Its 13-class outputs are not equivalent to the two-class model.
SRS UC13 Both files absent	The application raises FileNotFoundError during startup. The required model file must be restored before running inference.
SRS UC01–06 New training run	The developer runs the notebook in order and leaves the run-selection input empty. A separate output directory is created, and training starts from pretrained YOLOv8m weights.
Exception No target detected	If no helmet-related target passes the threshold, return an explicit no-detection message. Do not infer helmet compliance or switch to the backup model.

The updated predict() function rejects missing photo input with a clear message. detect() handles an empty camera frame and reports no helmet-related detection when no target is found. Corrupt files, interrupted cameras and end-to-end behaviour still require testing on the demonstration laptop.

1.7 Development Methodology

1.7.1 Chosen Methodology

The project follows the VU Process Model described in the SRS: structured Waterfall-style requirements, design and delivery stages, combined with Spiral-style risk review and iterative model refinement. The revised training and app work fit within these stages. Dataset quality, weak no-helmet recall and deployment constraints guide each review cycle.

The training split fits the model parameters. The validation split supports checkpoint selection and development choices. The test split estimates performance after those choices. Repeatedly choosing settings because they score well on the test set would make it a development set and weaken the final performance claim.

1.7.2 Reasons for Chosen Methodology

The hybrid approach preserves the SRS and design milestones while allowing dataset and model errors to be corrected iteratively. Changes are checked against validation evidence and the agreed requirements. Retaining YOLOv8m and the original checkpoint-loading order limits integration changes when the new weights and interface are deployed.

Separating notebook sections makes problems easier to locate. Data extraction, class conversion, image checks, augmentation, training and evaluation each have their own stage. The saved run directory keeps the evidence for a particular experiment together.

The approach also supports limited local hardware. The computationally demanding training runs on a Colab T4 GPU, while the application can perform inference in a compatible local Python environment. Colab runtime availability is variable, so checkpoints are stored on Drive and an interrupted run can be resumed when its training state remains available. [7]

A completed early-stopped run should be evaluated before beginning another experiment. More epochs alone do not guarantee improvement. If further revisions are needed, validation errors should guide data correction, and a fresh final test set should be reserved for any substantially revised system.

1.7.3 Work Plan (Gantt Chart)

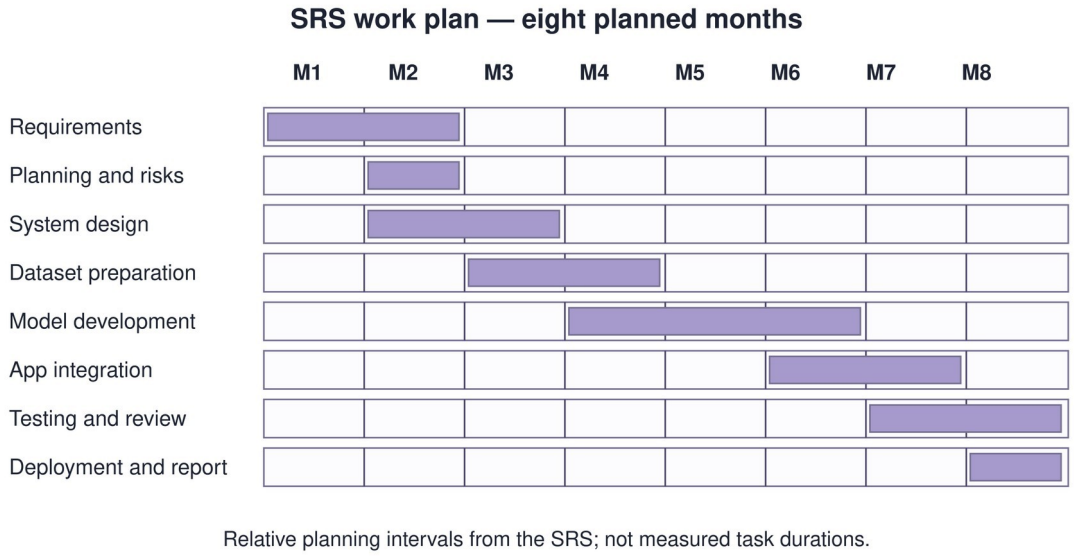


Figure 1.2 Relative work plan for the project

The chart follows the eight-month planning structure in the SRS, with overlapping requirements, design, training, integration and review activities. Month labels are planned relative intervals, not evidence of actual task duration. The dated implementation milestones below remain separate from that plan.

1.7.4 Project Schedule (Submission Calendar)

Milestone	Recorded date or scheduling basis
Dataset and notebook preparation	12 October 2026: reviewed data and notebook preparation in the project record.
Completed training run	13 October 2026: 36 epochs; selected checkpoint from epoch 16.
Package and result review	18 October 2026: packaged best.pt and evaluation report reviewed.
Final report review	19 October 2026: app and diagrams revised using SRS/design feedback.
Presentation preparation	After the group approves the report.
VULMS submission	Official deadline to be confirmed from the course submission calendar.

The submission instructions require the application, report and presentation in one folder, compressed as ZIP or RAR and submitted on VULMS. The supplied instructions do not state a due date. The team must use the course deadline rather than infer one from the archive name.

CHAPTER 2

Designing the Project

2.1 Introduction

The design separates model development from inference. The notebook creates a checkpoint and evaluation evidence; the app loads the checkpoint and processes photos or camera frames. Drive stores data and run outputs during development. The local application does not read the training dataset or Drive during ordinary inference.

2.2 Purpose

The design specifies the flow of images, labels, model weights and results. It identifies the software components that are present in the code and clarifies the boundaries between automatic processing and manual deployment. It also documents the file-based data model used instead of a database.

2.3 Scope

The design covers YOLOv8m, the existing YOLOv11 fallback, the Gradio photo/live-camera interface, direct OpenCV camera capture and Colab training. Updated diagrams follow the SRS and design structure. Proposed relational storage is identified separately because the running app has no database or persistent violation history.

2.4 Definitions, Acronyms and Abbreviations

Term	Design meaning
PIL image	RGB image used by the photo and camera detection functions.
Results	Ultralytics prediction object containing boxes, scores and class IDs.
YAML	Configuration file format used for split locations and class names.
JSON	Structured format used for settings, audits and evaluation values.
TXT label	YOLO text annotation: class ID and four normalised box coordinates.
Fingerprint	Digest used to check that resumed training uses the same prepared data.
Fallback	An alternative model selected at startup when the main file is absent.

2.5 Architectural Representation (Architecture Diagram)

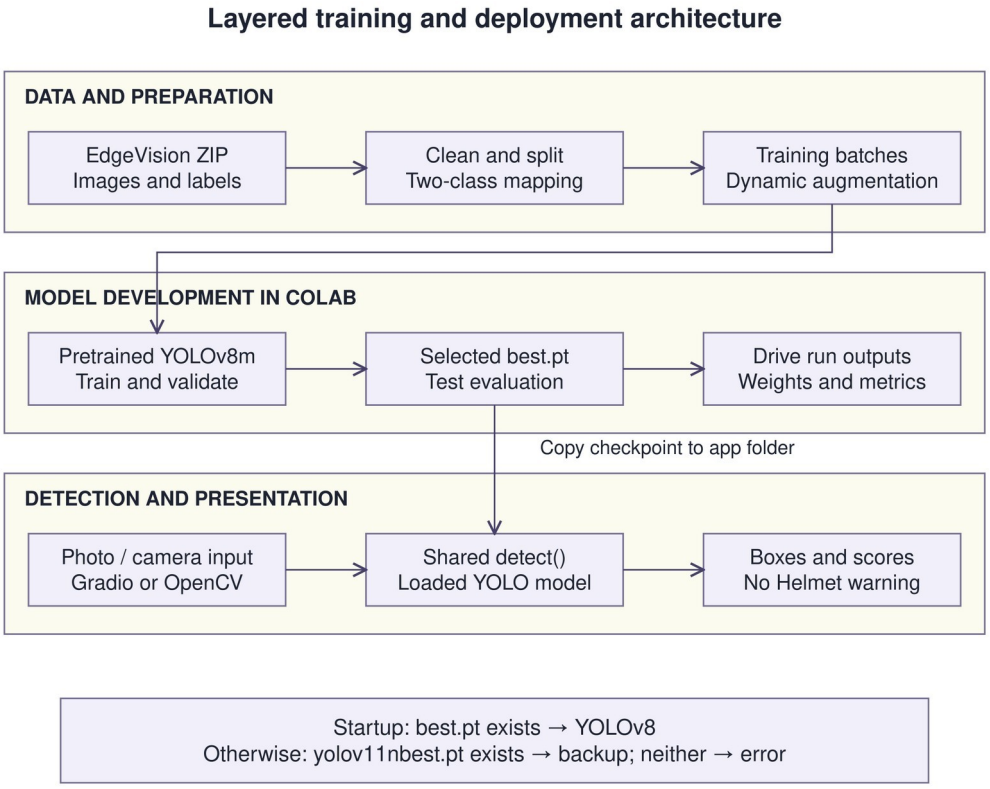


Figure 2.1 Training and application architecture

The upper stages run in Colab. Source files are read from the project folder, staged locally, cleaned and split. Training applies augmentation to training batches and saves checkpoints in the Drive run directory. Validation selects best.pt, and the test stage measures the selected model.

A developer copies best.pt beside app.py. Startup checks best.pt first and checks yolov11nbest.pt only if the main file is absent. Photo and browser-camera callbacks share detect(); the local OpenCV mode also uses it. Predictions are drawn in RGB with readable labels and scores. No dataset or API key is needed for this path.

2.6 Dynamic Model: Sequence Diagrams

2.6.1 Image and Webcam Detection Sequence

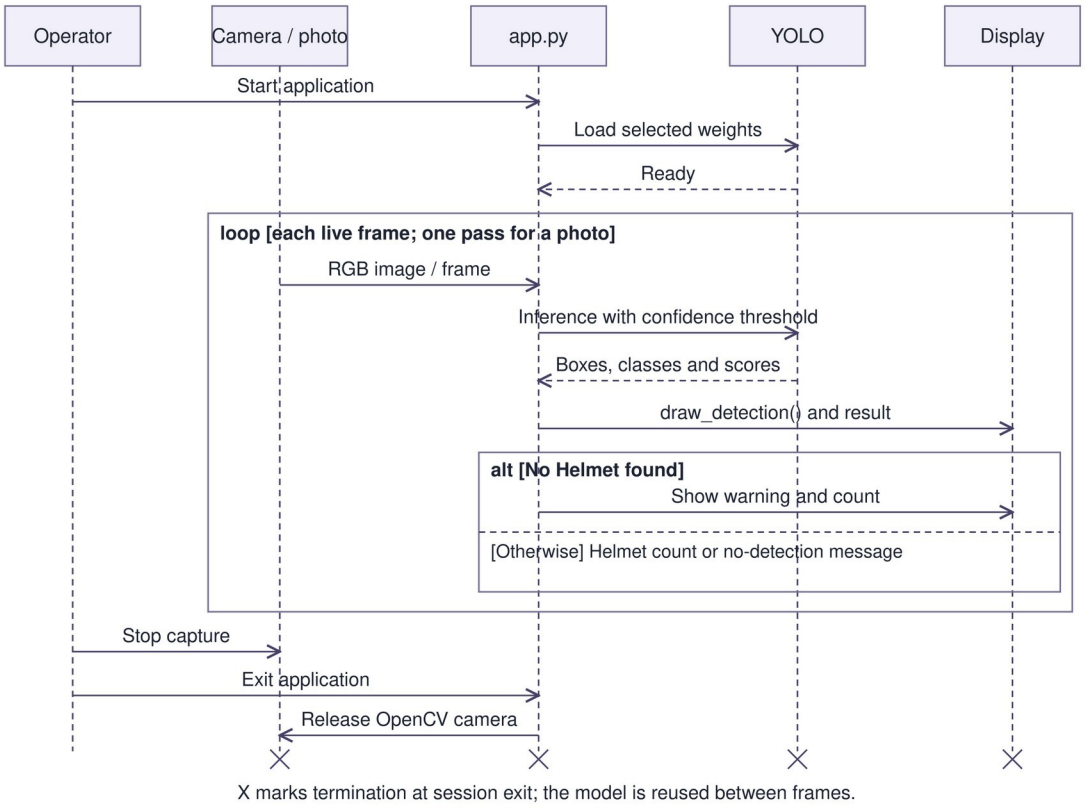


Figure 2.2 Photo and webcam processing with session termination

After startup loads the model, predict() processes a photo and stream_frame() processes browser camera frames through detect(). Direct OpenCV mode captures frames with VideoCapture and calls the same detection function. Each result contains boxes, class names and confidence scores; draw_detection() renders the annotated output.

The loop represents repeated camera frames; a photograph makes one pass. The conditional branch displays a warning when No Helmet is detected. Dashed arrows show returns and X markers show termination at session exit. Camera capture is released by the OpenCV path. No database log, saved violation or sound alert is implied.

2.6.2 Training and Evaluation Sequence

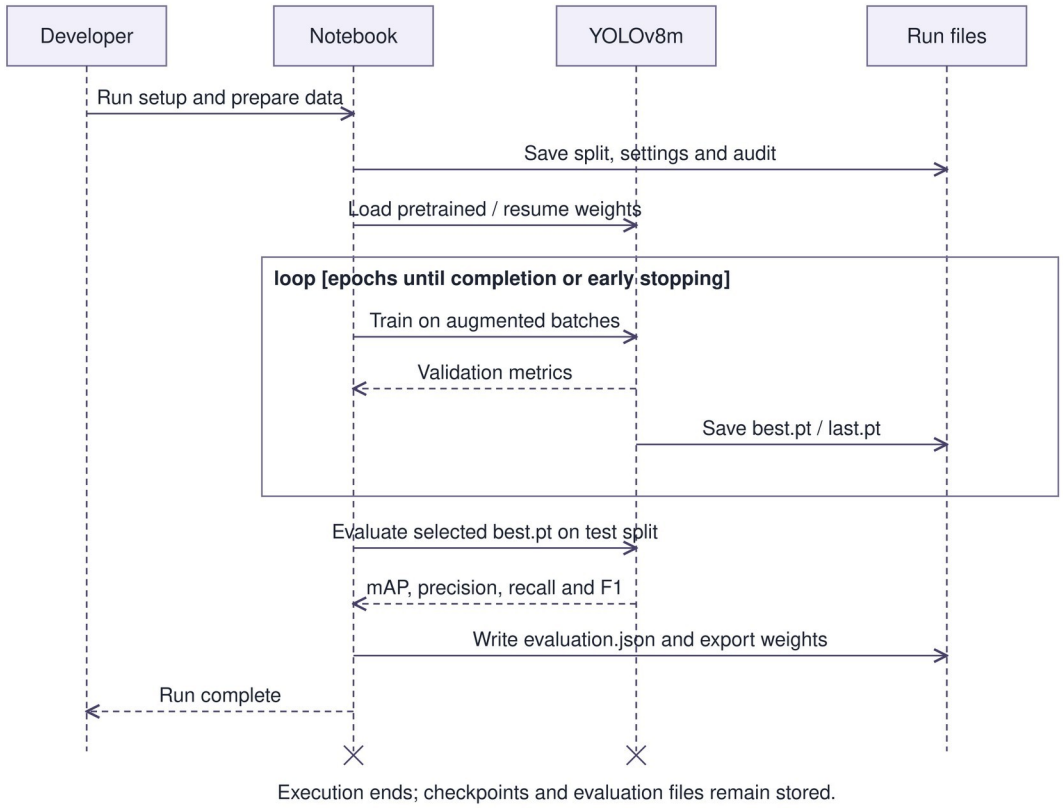


Figure 2.3 Training, checkpoint selection and evaluation

Before training, the notebook establishes the run directory, reads the dataset and constructs the prepared splits. A new run starts from official pretrained yolov8m.pt weights. A resumed run uses the saved training state when an eligible last.pt exists and the prepared dataset matches the saved fingerprint.

The trained model is selected by validation fitness. The test split is then evaluated without parameter updates. The notebook writes evaluation.json separately from the model checkpoint, allowing the group to inspect results without loading the neural network.

2.7 Object Model/Logical Model: Class Diagram

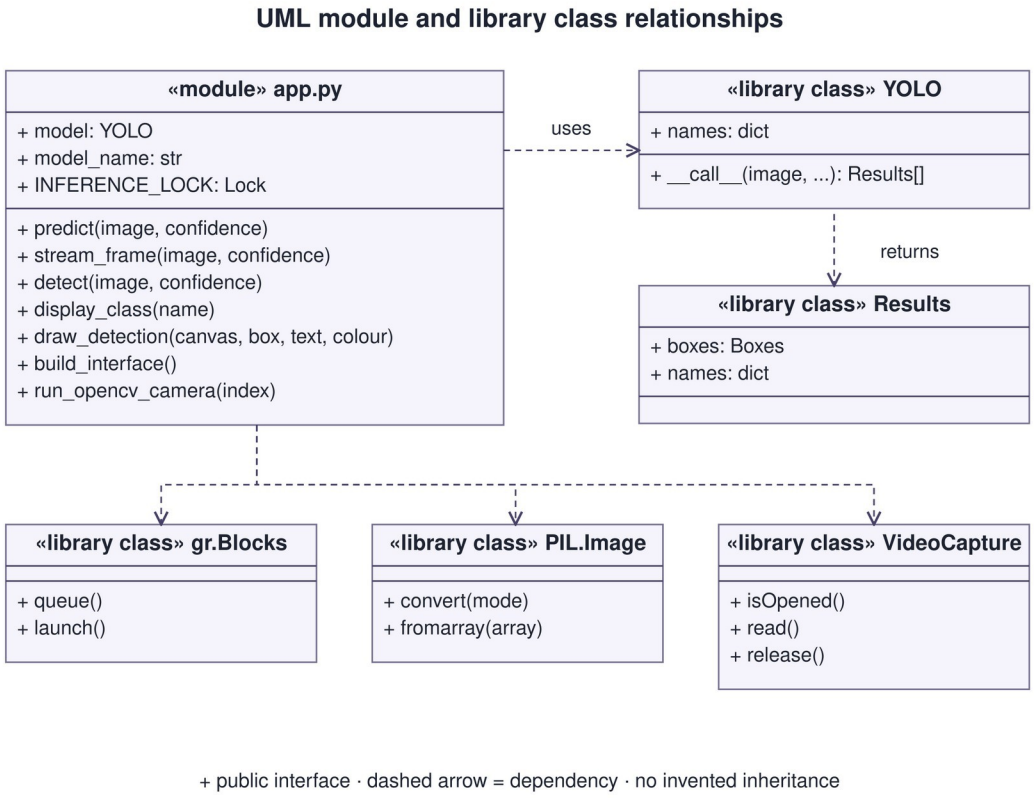


Figure 2.4 UML dependencies between app.py and library classes

The revised diagram uses separate name, attribute and operation compartments, public (+) visibility and labelled dependencies. app.py is marked as a module because it defines functions rather than a custom HelmetDetector class. Only relationships supported by the code are shown; inheritance and aggregation are not added without an implementation basis.

app.py stores model, model_name and an inference lock. predict(), stream_frame() and run_opencv_camera() call detect(). The module uses YOLO, Results, PIL.Image, gr.Blocks and OpenCV VideoCapture. The shared lock serialises model calls. display_class() maps names while draw_detection() controls box colours and text.

get_example_images() reads supported files from images/ for the Photo check tab. It does not score detections against annotations. Normalised class-name matching handles the backup’s helmet categories; unrelated classes such as person and motorbike keep their own labels and are not converted to No Helmet.

2.8 Database Model (Database Diagram)

The current implementation stores images, TXT labels, settings, weights and evaluation JSON as files. It has no SQL connection or persistent detection table. To clarify the earlier design, the following proposed relational representation uses explicit primary keys (PK), foreign keys (FK) and cardinalities; it is not an implemented database.

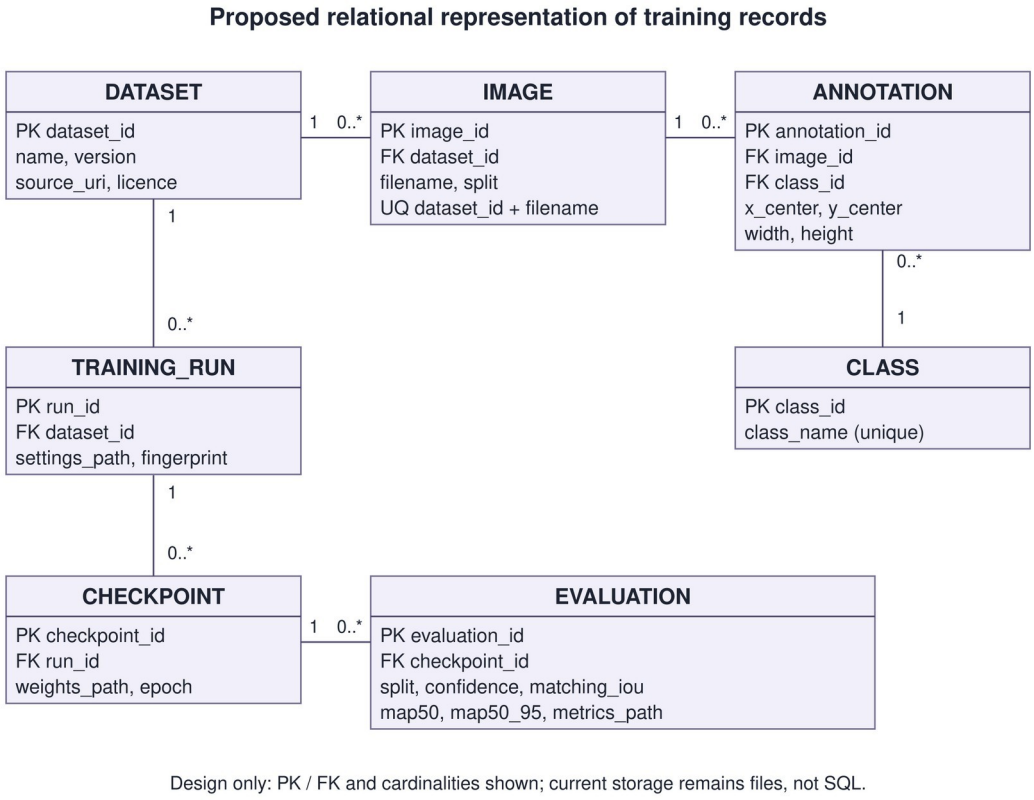


Figure 2.5 Proposed normalised data design with PK and FK relationships

Dataset metadata is stored once in DATASET. IMAGE references its dataset; ANNOTATION references an image and a class. TRAINING_RUN references the dataset, CHECKPOINT references its run and EVALUATION references a checkpoint. These one-to-many relationships avoid repeating dataset and class descriptions in every record.

In the delivered files, image and label pairs are linked by filename stem, data.yaml defines splits/classes and each run folder groups weights and metrics. Database migration would require implementation and testing. User accounts, violation records and sound alerts from the earlier design are not claimed as delivered features.

2.9 Graphical User Interfaces

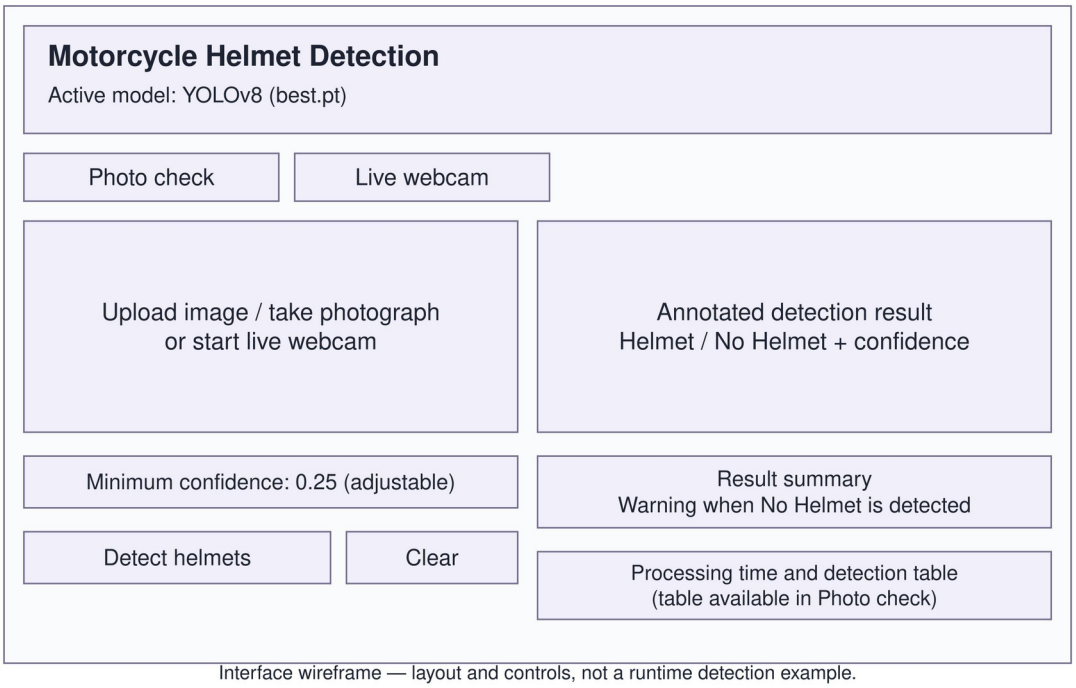


Figure 2.6 Updated photo and live webcam interface wireframe

The interface has Photo check and Live webcam tabs. Photo check accepts uploads or camera photographs and returns an annotated image, counts, confidence table and processing time. Live webcam updates annotated frames and status automatically. Both tabs expose a confidence slider, initially 0.25, and display the active model.

The app draws Helmet boxes in green and No Helmet boxes in red, with confidence percentages. No Helmet triggers an on-screen warning. Other backup classes use blue boxes. An empty result states that no helmet-related detection was found; it does not confirm helmet use. The captioned wireframe shows controls, not measured camera results.

Motorcycle-focused wording replaces the inherited construction-safety description. Browser camera access requires permission. Live sessions are limited to five minutes where supported by Gradio and can be restarted. Actual update speed depends on the computer; no laptop frame-rate measurement is reported.

The command python app.py --webcam opens a direct OpenCV camera window on the local computer. Q or Esc stops capture; closing the window also exits. python app.py starts the browser interface. Final screenshots and webcam acceptance results should be captured on the demonstration device.

CHAPTER 3

Development

3.1 Development Plan (Architecture Diagram)

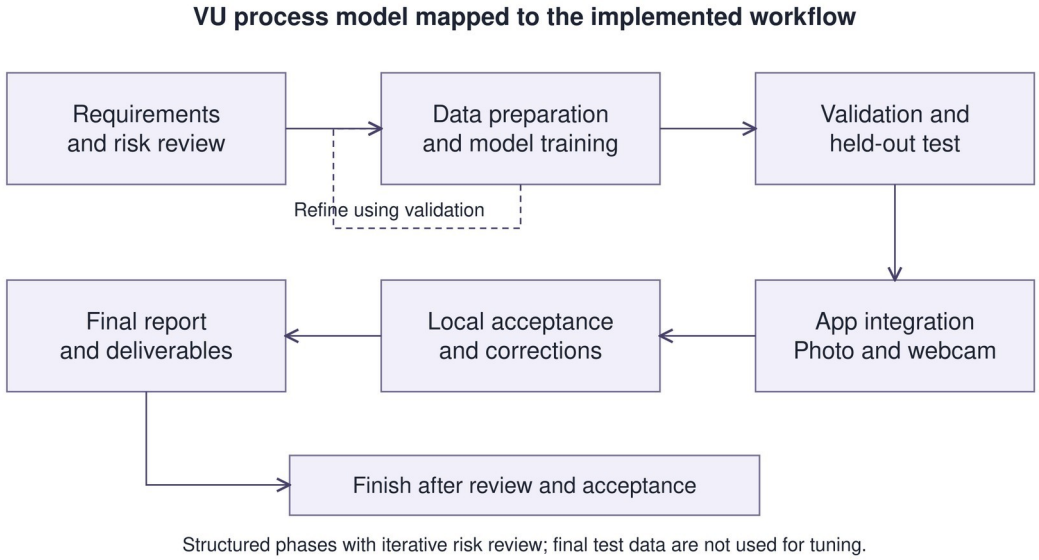


Figure 3.1 Development and verification stages

The implementation retains YOLOv8m and its two output classes. Twenty notebook sections cover Drive data preparation, training and evaluation. The revised app extends the final integration stage with photo and live-camera paths, readable labels and a warning message. Validation and local acceptance remain distinct stages.

Sections	Implementation responsibility
1–4	Install libraries, connect Drive, select a run, define the model and settings.
5–9	Read the complete dataset, convert labels, clean images and save an audit.
10–12	Group related images, create splits, balance exposure and preview labels.
13–15	Configure augmentation, train or resume, and display training evidence.
16–18	Evaluate the held-out test split and optionally compare an old checkpoint.
19–20	Download model/report files and explain application integration.

The current trained checkpoint and the included notebook correspond to the run edgevision_20261013_091419_146922. The source architecture has 25,840,918 parameters. The exported checkpoint is approximately 52 MB; a weight file is the trained model artifact, whereas the notebook is the code that produces it.

3.2 Dataset Acquisition and Class Conversion

EdgeVision V1 supplies 2,392 images and annotations in several formats. The project uses its YOLO TXT labels. The source class list contains BikeWithRider, NoHelmet and Helmet. The README and licence are retained with the training material, and the dataset is attributed to its creators. [3]

Source ID and label	Action	Output ID and label
0 BikeWithRider	Discard this box	Not an output class
1 NoHelmet	Retain and remap	1 non-Helmet-
2 Helmet	Retain and remap	0 accept-Helmet-

Dropping a BikeWithRider box does not require dropping the image. If the image also has valid head labels, those labels remain. Images with no retained target boxes are excluded rather than treated automatically as verified negative examples.

Checkpoint labels remain accept-Helmet- and non-Helmet- for compatibility. The app displays them as Helmet and No Helmet without changing the weights. The detector predicts head regions in motorcycle scenes; it does not explicitly associate a head with a motorcycle or verify that every detected person is a rider.

The full dataset ZIP and notebook are placed in My Drive/Helmet_Project. The loading code finds supported ZIP files and extracts only the required images, labels and metadata into the Colab working area. Extraction happens automatically; the developer does not need to unzip manually in Colab. The source files on Drive are retained.

The completeness check requires all 2,392 source images and corresponding labels. The previously reviewed 209-image sample and the separate labels-only archive are not complete training datasets. The final evaluation report confirms that the completed run began from 2,392 source images. [2]

3.3 Data Cleaning and Validation

Cleaning is performed before splitting and training. The label reader checks that each retained row has the expected class and four finite box coordinates. Target boxes with malformed or non-positive area cause the affected image to be rejected. Boxes crossing the image edge are clipped, and duplicate annotation rows are removed.

Images must decode successfully and agree with the available metadata dimensions. The pipeline checks orientation and preserves supported upright cases. Images are resized proportionally to a maximum longest side of 1,600 pixels and saved at JPEG quality 95. Proportional resizing leaves normalised YOLO coordinates valid.

Check	Purpose and action
Exact image duplication	Compare decoded pixels; retain one identical labelled copy and exclude conflicting duplicate-label groups.
Oversized target boxes	Quarantine boxes wider or taller than 60% of the image for review; some may be legitimate close-ups.
Confirmed annotation defects	Exclude Image_01355, Image_01384, Image_01441 and Image_01512 based on prior visual review.
Missing or incorrect semantics	Require visual review; numeric checks cannot determine whether every visible head has the correct label.
Audit output	Record exclusions and transformations so the preparation decisions can be inspected.

The completed run retained 2,330 unique images from the 2,392 source images. The reduction of 62 images reflects the combined preparation filters; the evaluation report alone does not break those 62 into individual causes.

Only 209 source images received visual review before notebook construction. The automated checks do not certify that all remaining annotations are correct. The preview in section 12 is a spot check and is particularly useful for finding boxes around motorcycles instead of heads or visible unhelmeted heads without labels.

3.4 Dataset Splitting and Training Exposure

Related images are grouped before the approximately 70/15/15 split. The grouping combines perceptual-image similarity and conservative blocks of consecutive numbered images. An optional capture_groups.csv can specify capture groups when their identities are known. These measures reduce obvious leakage but do not prove that all images from the same scene or video remain together.

Split	Unique images	Helmet boxes	No-helmet boxes
Training	1,642	1,140	1,931
Validation	340	234	408
Test	348	231	408
Total	2,330	1,605	2,747

The split assignment is selected using image and class counts rather than model test results. Each original image belongs to one split. Validation and test images are not duplicated to balance class counts. A dataset fingerprint supports consistency checking when a training run is resumed.

In this dataset, helmet boxes are the minority within the training split. The training-only balancing stage repeats selected minority-containing images, with at most two additional copies per selected image. It increases exposure without presenting those copies as new independent examples.

Training measure	Helmet	No helmet
Original labelled boxes	1,140	1,931
Boxes across training exposures	1,889	1,988

Training uses 2,233 image exposures derived from 1,642 unique training images. Both classes can be present in the same image, so class balancing cannot be described simply as an equal number of single-class photographs. Dynamic augmentation further varies the appearance of training batches without increasing the count of independent source images.

3.5 Data Augmentation

The notebook explicitly configures Ultralytics augmentation before model.train(). During training, random transforms alter image appearance and geometry while the corresponding bounding boxes are transformed with the images. The settings are recorded in augmentation.json and the training arguments. [1, 6]

Setting	Value	Intended variation
hsv_h / hsv_s / hsv_v	0.015 / 0.4 / 0.3	Hue, saturation and brightness changes.
degrees	5	Small rotations.
translate	0.1	Position shifts within the image.
scale	0.3	Changes in apparent size.
fliplr	0.5	Horizontal flipping probability.
mosaic	0.5	Mosaic augmentation probability.
flipud	0	No vertical flipping.
shear / perspective	0 / 0	These geometric transforms are disabled.
mixup / copy_paste	0 / 0	These augmentation methods are disabled.
close_mosaic	10	Configured to disable mosaic in the final 10 planned epochs.

Training was planned for 100 epochs and stopped at epoch 36. Therefore, the final ten planned epochs were not reached; the configuration should not be described as proof that mosaic was disabled during the last ten epochs of this actual run.

The augmented images are generated dynamically. The project does not need a permanent folder containing every transformed image. Saved train_batch*.jpg files show example batches actually used during training, and augmentation.json plus args.yaml document the configured operations. These files can be shown during the viva.

Validation and test images do not receive random training augmentation. Augmentation does not correct a missing or wrong label, and it does not guarantee performance on new camera conditions. The dataset being loaded from a ZIP rather than an API does not disable augmentation.

3.6 Model Training and Checkpoint Selection

Parameter	Configured value
Starting model	Official pretrained yolov8m.pt
Output classes	2; accept-Helmet- and non-Helmet-
Image size / batch size	640 / 8
Maximum epochs / patience	100 / 20
Optimiser	AdamW
Initial learning rate / final factor	0.001 / 0.01
Weight decay	0.0005
Seed / deterministic flag	42 / True
Workers / mixed precision	2 / AMP enabled
Training environment	Tesla T4; Ultralytics 8.3.162

Training uses transfer learning. The architecture starts with pretrained weights and learns the two target classes from the prepared dataset. It is not a new neural-network architecture trained from random weights. The final log records Python 3.13.15 and PyTorch 2.11.0+cu130 in the Colab environment. [1, 4]

The completed run took 1.044 hours for 36 epochs. Validation fitness was best at epoch 16. Early stopping ended the run after 20 further epochs without improvement. The exported best.pt therefore represents the selected checkpoint, not necessarily the final epoch’s parameters.

Weights are saved to the run’s training/weights directory. best.pt is used for inference; last.pt records the most recent training state while training is active. After completion, Ultralytics strips optimizer data from the final exported weights. A completed stripped checkpoint should not be treated as an interrupted checkpoint with full resume state.

The notebook also copies best.pt to the top level of the run directory. This is the file downloaded for the app. The separate notebook-saving warning encountered in Colab concerns saving notebook edits and outputs; it is distinct from the successful model-save messages in the training log.

3.7 Evaluation Method and Results

The selected checkpoint was evaluated on 348 test images containing 231 helmet and 408 no-helmet annotations. Ultralytics produced AP-based metrics. A separate class-aware one-to-one matcher measured precision, recall and F1 at confidence 0.25, matching IoU 0.50 and NMS IoU 0.70. [2, 5]

Test measure	Helmet	No helmet
Correct detections (TP)	209	348
Incorrect detections (FP)	53	77
Missed targets (FN)	22	60
Precision	79.77%	81.88%
Recall	90.48%	85.29%
F1	84.79%	83.55%
AP50	92.82%	88.84%
AP50-95	66.57%	55.32%

Overall test mAP50 is 90.83%; mAP50-95 is 60.95%. The stricter measure assesses a range of overlap thresholds and is lower because precise localisation is harder. Neither value is a generic classification accuracy. The app-threshold measurements count errors at one chosen confidence setting.

Precision = TP / (TP + FP). Recall = TP / (TP + FN). F1 = 2 × precision × recall / (precision + recall). For no helmet, 348 / 408 gives 85.29% recall, and 348 / 425 gives 81.88% precision.

The no-helmet precision and recall exceed the project’s 75% acceptance target on this protocol. The result still includes 60 missed targets and 77 false detections. The report’s approximate recall interval is 81.53%–88.40% for no helmet, but related images and heads weaken the independence assumption behind that interval.

The original model reported 58.7% validation mAP50 and 29.6% no-helmet recall on 85 different validation images. The new model reported 91.9% validation mAP50 and 82.6% no-helmet recall on 340 images. These support further comparison, but do not establish the size of improvement on an identical test set.

3.8 Testing and Verification

Test	Evidence or outcome
Full-dataset preparation	evaluation.json records 2,392 source images and 2,330 retained unique images.
Split sizes	1,642 training, 340 validation and 348 test images are recorded.
Two-class compatibility	Checkpoint metadata contains class 0 accept-Helmet- and class 1 non-Helmet-.
Training completion	Recorded log confirms 36 completed epochs and selection of epoch 16.
No-helmet target	Test precision 81.88% and recall 85.29% both exceed 75%.
Model selection route	Original loading block preserved; priority and missing-file branches checked with a simulated detector.
Local end-to-end app run	Pending verification on the intended demonstration computer.
Continuous camera detection	Browser streaming and direct OpenCV capture implemented; physical-camera acceptance pending.
Labels, colour rendering and empty input	Checks with simulated predictions passed. Corrupt-file and device tests remain pending.

App checks used simulated model outputs to exercise loading priority, missing-weight errors, label mapping, RGB preservation, empty detections and interface construction. They did not evaluate the trained network or a physical webcam. Deployment-device acceptance and timing tests are still required.

Run the SRS camera test for five minutes on the intended laptop. Include helmeted and unhelmeted heads, multiple people, occlusion and no-target frames. Test camera permission refusal, camera disconnection and stop controls. Record latency, missed heads and false detections alongside fresh motorcycle-image checks.

The optional old-versus-new comparison requires best_original.pt and identical annotated test images and thresholds. Its result is not present in evaluation.json. The requirement “compare both models” does not identify the second model; the supervisor must confirm the intended comparison before this item is considered complete.

3.9 Application Integration and Deployment

Extract the project and replace app.py with the revised file. Keep best.pt and yolov11nbest.pt beside it. Run python app.py from that folder, since the preserved loader uses relative paths. Use the Photo check or Live webcam tab; allow camera permission when requested. Restart the process after changing weights.

File	Role during deployment
app.py	Loads the model; provides photo, browser webcam and local OpenCV modes.
best.pt	Primary trained two-class YOLOv8m checkpoint.
yolov11nbest.pt	Existing alternative checkpoint used only when best.pt is absent.
requirements.txt	Verify Ultralytics, Gradio, Pillow, NumPy and OpenCV; desktop camera mode needs GUI-enabled OpenCV.
images/	Example images displayed by the app.
evaluation.json	Evidence for test results; not required by the inference function.
Training notebook and dataset	Used for training and reproducibility, not required for ordinary app inference.

The old best.pt may be retained under a different name for comparison or recovery. That backup is optional for inference. Renaming it does not cause both models to run together. The main route remains active whenever a file named best.pt exists.

The app does not inspect a prediction and then switch to the fallback when it is uncertain. A load failure in an existing best.pt is also not handled by the file-absence branch. The fallback is therefore a startup convenience, not an ensemble or a guarantee of detection quality.

Use python app.py --webcam for direct OpenCV capture; --camera 1 selects an alternative camera. The desktop mode requires OpenCV with GUI support. The browser interface uses compatible Gradio styling for versions 4/5 and 6. RGB colour checks passed with simulated detections; record the final dependency versions and laptop timings.

3.10 Limitations, Future Work and Conclusion

The key measured result is no-helmet detection on the revised dataset's held-out split. At the app threshold, the model detects about 85 of every 100 annotated no-helmet heads in that test set. The remaining misses and false detections matter for any practical deployment.

The dataset grouping is heuristic. It cannot guarantee that every related capture is isolated within one split. Only part of the source collection received visual annotation review, so remaining missing or incorrect labels may affect training and scoring. The final test set should not be repeatedly reused to select further model changes.

The two-class detector operates on visual head regions. It does not explicitly prove that a person is riding a motorcycle or distinguish every worn helmet from a carried helmet in an unfamiliar scene. It also does not establish robust performance for indoor laptop selfies, night scenes, severe blur or camera viewpoints absent from the data.

Further work should prioritise annotation review, independent road and webcam tests, and measured frame rate, latency and stability. Continuous capture is now implemented, but field performance has not been established. Motorcycle/head association, persistent incident storage, sound alerts and recorded-video upload remain future extensions.

The project delivers a YOLOv8m training notebook, two-class checkpoint, evaluation record and updated photo/webcam app. The selected checkpoint is from epoch 16 of a 36-epoch run. Test no-helmet precision is 81.88% and recall is 85.29% under the documented protocol. The original loading route is preserved; deployment acceptance remains a separate check.

REFERENCES

[1] Project implementation records. HELMETY8_v4.ipynb, HELMETY8_v5_EdgeVision.ipynb, requirements.txt and checkpoint metadata in the project archive; revised app.py, 19 October 2026. Training run edgevision_20261013_091419_146922, 13 October 2026. App checks used simulated predictions.

[2] Project evaluation report. evaluation.json for the two-class EdgeVision run, supplied 18 October 2026. Contains split counts, AP metrics and app-threshold TP, FP, FN, precision, recall and F1.

[3] Gajjar, K., Patel, N., Patel, R., Patel, K., Dabhi, C., Trivedi, H., Goyani, M. and Bhatt, C. (2026). EdgeVision Dataset. Mendeley Data, V1. DOI: 10.17632/j82bnw7gsr.1. https://data.mendeley.com/datasets/j82bnw7gsr/1. Licence: CC BY 4.0. Preparation changes include class conversion, cleaning, resizing and splitting.

[4] Ultralytics. Model Training with Ultralytics YOLO. https://docs.ultralytics.com/modes/train/. Accessed 18 October 2026. The project configuration and checkpoint use version 8.3.162; current documentation may describe later releases.

[5] Ultralytics. Model Validation with Ultralytics YOLO. https://docs.ultralytics.com/modes/val/. Accessed 18 October 2026.

[6] Ultralytics. Data Augmentation using Ultralytics YOLO. https://docs.ultralytics.com/guides/yolo-data-augmentation/. Accessed 18 October 2026.

[7] Google. Colaboratory Frequently Asked Questions. https://research.google.com/colaboratory/faq.html. Accessed 18 October 2026.

[8] Virtual University of Pakistan. Supplied CS619 final-deliverable template and submission instructions; SRS-Doc Spring 2026_CS619; Design-Doc Spring 2026_CS619; marked design feedback Mrk_Spring 2026_CS619_13354_43920_S26PROJECT7DE39. Group S26PROJECT7DE39. These records guide the report structure and diagram revisions.

APPENDIX

A Project Files and Reproducibility Record

Item	Recorded value or location
Run identifier	edgevision_20261013_091419_146922
Drive project folder	My Drive / Helmet_Project
Run folder	outputs / edgevision_20261013_091419_146922
Primary downloadable checkpoint	Run folder / best.pt
Training checkpoints	Run folder / training / weights / best.pt and last.pt
Evaluation report	Run folder / evaluation.json
Augmentation configuration	Run folder / augmentation.json
Training argument record	Run folder / training / args.yaml
Example augmented batches	Run folder / training / train_batch*.jpg
Training curves	Run folder / training / results.png
Dataset licence and attribution	Source README and LICENSE metadata

Dataset fingerprint recorded in evaluation.json:

96863c908f4020c2d2d2e835b2ee8d1f05ee6b2965cb379285bdbc72bd4b3a86

The application archive still contains data.yaml for the earlier Roboflow dataset. It is not the configuration used for the new EdgeVision training run. The revised notebook generates its own data.yaml inside the prepared run dataset. The app’s inference function does not read either dataset YAML file.

Keep the source ZIP, notebook, selected weights, evaluation report and preparation audits together for reproducibility. A model checkpoint alone cannot document which data were excluded, how related images were grouped or which evaluation thresholds were used.

B Core Application Code

These excerpts show the preserved startup selection and the revised photo callback. The loaded model is reused by the shared detect() function for photo and camera processing. The full app.py contains the drawing, streaming and OpenCV capture functions.

```
if os.path.exists("best.pt"):
    model = YOLO("best.pt")
    model_name = "YOLOv8 (best.pt)"
elif os.path.exists("yolov11nbest.pt"):
    model = YOLO("yolov11nbest.pt")
    model_name = "YOLOv11 (yolov11nbest.pt)"
else:
    raise FileNotFoundError(
        "No model file found. Please ensure 'best.pt' "
        "or 'yolov11nbest.pt' exists."
    )

def predict(image, confidence, progress=gr.Progress()):
    if image is None:
        raise gr.Error("Please upload an image or take a photograph first.")
    progress(0.1, desc="Checking image...")
    output = detect(image, confidence)
    progress(1, desc="Detection complete")
    return output
```

detect() converts the input to RGB, calls the model with the chosen confidence and IoU 0.70, maps class names and draws boxes. It returns the annotated image, status, detection rows and processing time. The model lock prevents concurrent callbacks from using the predictor simultaneously.

stream_frame() returns the image, status and timing for browser video. run_opencv_camera() reads frames, converts BGR to RGB for detection, then converts the rendered output back to BGR for display. A finally block releases the camera. Display changes do not retrain or modify best.pt.

C Operation and Demonstration Notes

Training preparation: place the full EdgeVision ZIP and the revised notebook in My Drive/Helmet_Project. Open the notebook in Colab, select a T4 GPU and run the numbered sections in order. Section 3 accepts an empty response for a new run or a listed run number for resuming an eligible checkpoint.

Training completion: early stopping is a normal completion condition. Inspect the selected model's validation results and then run the test evaluation. Section 19 downloads best.pt and evaluation.json to the browser's configured download location. The files also remain in the Drive run folder.

Application use: keep the updated app.py and best.pt in the extracted project folder. Run python app.py for Photo check and Live webcam. Alternatively, run python app.py --webcam for direct OpenCV capture; press Q or Esc to stop. Confirm labels, colours, confidence values and camera behaviour on the actual laptop.

Augmentation demonstration: show augmentation.json or args.yaml to explain the configured transforms, then show saved train_batch*.jpg examples. State that augmentation is produced dynamically during training and that only sample batches are saved as images. Do not claim that all augmented files exist in a separate permanent dataset.

Result explanation: report the held-out test size and thresholds before quoting performance. For no helmet, precision is 81.88% and recall is 85.29% at confidence 0.25 and matching IoU 0.50. Overall test mAP50 is 90.83%. Avoid calling all three values "accuracy" because they measure different aspects of detection.

Submission preparation: include the application, final report and approved PPT in the VULMS ZIP/RAR, with model weights and dependencies. The marked design feedback also requires the corrected design document with the final deliverable. Database diagrams are proposed designs; no database file exists in the current application.